

■ GitFlow & CI/CD Pipeline Completo

Última atualização: 12/03/2026

■ GitFlow & CI/CD Pipeline

Visão Geral

Este projeto segue um fluxo simplificado baseado em GitFlow com foco em: Qualidade (quality gates automáticos), Segurança (scanning de vulns), Automação (deploy automático), e Rastreabilidade (commits atômicos).

■ Branch Strategy

main

■ Produção. Auto-deploy em push. Merge protegido. Always stable.

develop

■ Integração. Destino de todos os PRs. Pre-produção.

staging

■ Teste. Ambiente idêntico à produção antes de liberar main.

feature/*

Novas features. Temporário, deletado após merge.

■ Tipos de Branches Temporários

feature/short-description - Novas features

fix/short-description - Bug fixes

docs/short-description - Documentação

refactor/short-description - Refactoring

test/short-description - Testes

chore/short-description - Manutenção

■ Quality Standards

ESLint (sem erros) - Code style

Prettier (formatado) - Consistent formatting

TypeScript (sem erros) - Type safety

Tests (70%+ coverage) - Functionality

SonarCloud (Grade A-) - Code quality

npm audit (sem HIGH/CRITICAL) - Dependency security

Secrets scan (nenhum) - No exposed secrets

License check (MIT/Apache/BSD) - Legal compliance

Docker build (success) - Image buildable

1+ review approval - Human validation

■ Auto-Deploy Trigger

Quando?

Push para main branch

O quê?

GitHub Actions workflow deploy-to-vm.yml

Processo

1) Build Docker image 2) Push para GCP 3) Deploy via gcloud SSH 4) Health checks 5) Create release tag

Tempo

~7 minutos do push até produção ao vivo!

■ Troubleshooting

ESLint falha?

npm run lint --fix && npm run format

TypeScript falha?

npm run build && corrigir tipos manualmente

Testes falham?

npm run test --coverage && adicionar testes

npm audit?

npm audit fix && atualizar deps

Deploy falha?

gh run view --log && investigar